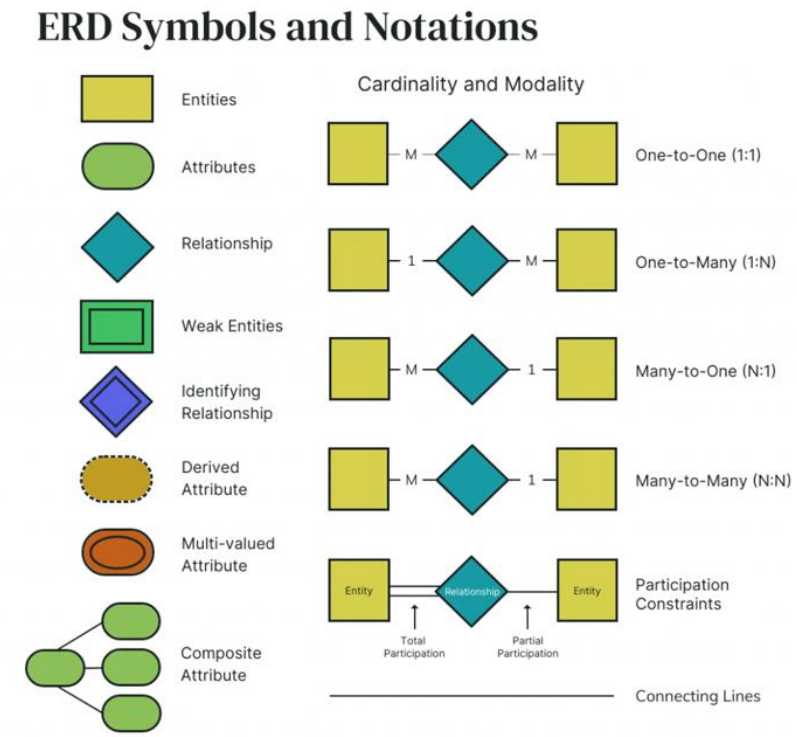
SQL Reference

Command	Description	Syntax	Example
CREATE / ALTER TABLE	Creates a table, or adds a column to an existing table	CREATE TABLE t (col type, ...); ALTER TABLE t ADD COLUMN col type;	CREATE TABLE s (id SERIAL PRIMARY KEY, name VARCHAR(50) NOT NULL);
Data types	INT; SERIAL (auto-incrementing integer); NUMERIC(p,s) exact decimal; VARCHAR(n); TEXT; DATE; BOOLEAN	col_name type	price NUMERIC(8,2)
Constraints	PRIMARY KEY (also composite), NOT NULL, DEFAULT, FOREIGN KEY	col type REFERENCES t(col) PRIMARY KEY (a, b) col type DEFAULT value	dept_id INT REFERENCES dept(dept_id); PRIMARY KEY (s_id, c_id)
INSERT / UPDATE / DELETE	Adds rows, changes values, removes rows (always use WHERE with UPDATE and DELETE)	INSERT INTO t (c1, c2) VALUES (v1, v2); UPDATE t SET c = expr WHERE cond; DELETE FROM t WHERE cond;	UPDATE s SET age = age + 1 WHERE id = 3;
SELECT / DISTINCT / AS / LIMIT	Chooses columns, removes duplicate rows, renames an output column, limits the number of rows	SELECT DISTINCT c AS alias FROM t LIMIT n;	SELECT DISTINCT age AS a FROM s LIMIT 5;
WHERE operators	Filters rows: =, <>, <, >, AND, OR, NOT, IS NULL, BETWEEN (inclusive), IN, LIKE (% = any text, _ = one character)	WHERE c BETWEEN a AND b WHERE c IN (v1, v2) WHERE c LIKE 'A%'	WHERE age BETWEEN 20 AND 25 AND email IS NOT NULL
ORDER BY	Sorts the result; ASC is the default; NULLS FIRST / NULLS LAST places NULL values	ORDER BY c1 DESC, c2 ASC	ORDER BY age DESC, name
INNER JOIN / LEFT JOIN	INNER keeps matching rows only; LEFT keeps every row of the left table (NULL when there is no match)	FROM a JOIN b ON a.k = b.k FROM a LEFT JOIN b ON a.k = b.k	FROM s LEFT JOIN c ON s.id = c.s_id WHERE c.s_id IS NULL
Self join	Joins a table to itself using two aliases	FROM t x JOIN t y ON x.fk = y.pk	FROM s a JOIN s b ON a.mentor_id = b.id
Subquery / EXISTS	A query inside parentheses; EXISTS is true when the subquery returns at least one row	WHERE c > (SELECT AVG(c) FROM t) WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.k = t1.k)	WHERE age > (SELECT AVG(age) FROM s)
WITH (CTE)	A temporary named result that can be used inside one query	WITH name AS (SELECT ...) SELECT ... FROM name;	WITH a AS (SELECT dept, AVG(age) AS av FROM s GROUP BY dept) SELECT * FROM a;
UNION / INTERSECT / EXCEPT	Combines two queries that have the same columns: all distinct rows / rows in both / rows of the first that are not in the second	q1 UNION q2 q1 INTERSECT q2 q1 EXCEPT q2	SELECT name FROM s EXCEPT SELECT name FROM t

Command	Description	Syntax	Example
COUNT / SUM / AVG / MIN / MAX / ROUND	Aggregate functions (they ignore NULL, COUNT(*) counts rows); ROUND(x, n) rounds to n decimals	COUNT(*), AVG(c), ROUND(AVG(c), 1)	SELECT ROUND(AVG(age), 1) FROM s;
GROUP BY / HAVING	Groups rows for aggregates; WHERE filters rows before grouping, HAVING filters groups after it	SELECT c, COUNT(*) FROM t WHERE cond GROUP BY c HAVING COUNT(*) > 1	SELECT dept, COUNT(*) FROM s GROUP BY dept HAVING COUNT(*) > 2;
OVER / PARTITION BY	Window function: calculates across related rows without collapsing them	func() OVER (PARTITION BY c1 ORDER BY c2)	AVG(age) OVER (PARTITION BY dept)
ROW_NUMBER / RANK / DENSE_RANK	Unique number / rank with gaps after ties (1, 1, 3) / rank without gaps (1, 1, 2)	RANK() OVER (ORDER BY c DESC)	ROW_NUMBER() OVER (ORDER BY age DESC)

ER Notation



Question 1: Library Database (50 points)

Use PostgreSQL syntax. The database has three tables (PK = primary key, FK = foreign key):

```
authors(author_id PK, author_name, country)
books(book_id PK, title, pages, isbn, price, author_id FK, recommended_id FK, publish_date)
audiobooks(audiobook_id PK, title, price)
```

books.author_id refers to authors.author_id, and books.recommended_id refers to books.book_id (the book recommended as the next read).

The tables contain this data:

```
INSERT INTO authors VALUES
  (10, 'Elchin', 'Azerbaijan'), (20, 'Mina', 'Azerbaijan'),
  (30, 'Jane', 'UK'), (40, 'Tomas', 'Germany'), (50, 'Hana', 'UK');

INSERT INTO books VALUES
  (1, 'The Silent River', 410, '978-0-111', 40.00, 10, NULL, '2015-03-01'),
  (2, 'Winter Roads',     380, '978-0-112', 30.00, 10, 1,    '2018-06-15'),
  (3, 'Dust and Gold',    290, NULL,        25.00, 10, 2,    '2020-01-10'),
  (4, 'The Last Harbor',  330, '978-0-114', 22.00, 20, 1,    '2019-09-01'),
  (5, 'Paper Cities',     270, '978-0-115', 22.00, 20, 4,    '2021-04-20'),
  (6, 'Night Letters',    360, NULL,        35.00, 30, 1,    '2016-11-05'),
  (7, 'Small Fires',      240, '978-0-117', 20.00, 30, 6,    '2022-02-14'),
  (8, 'Iron Gardens',     520, '978-0-118', 55.00, 40, 1,    '2012-08-30'),
  (9, 'Glass Mountain',   310, '978-0-119', 30.00, 40, 8,    '2020-07-07'),
  (10,'Blue Hours',       320, '978-0-120', 25.00, NULL, 1,  '2019-05-12');

INSERT INTO audiobooks VALUES
  (1, 'Small Fires', 15.00), (2, 'Quiet Maps', 18.00),
  (3, 'Glass Mountain', 22.50), (4, 'Salt Roads', 12.00);
```

1. Draw ER diagram. Use Entity, Attributes, Relationships and Cardinality.

2. Create and change data (DDL and DML). Write the statements for:
    a. a table `editions` with `edition_id` (auto-incrementing primary key), `label` (text up to 60 characters, required), `print_cost` (number with 2 decimals, up to 10 digits), `release_date` (date) and `book_id` (foreign key to `books`);
    b. adding a column `language` (up to 20 characters, default `'English'`) to `editions`;
3. Select, filter and sort.
    a. Return `book_title` (the title), `price` and `publish_date` of books written by author 10, 20, 30 or 40, priced between 20 and 40 (both included), that have an ISBN (not NULL) and whose title does not start with The. Show the highest price first, equal prices by title from A to Z, and only the first 3 rows.
    b. Return each country where an author comes from once, in alphabetical order.
4. Joins.
    a. Return `title`, `author_name` and `country` of every book that has an author, ordered by title.
    b. Return the `author_name` of every author that has no books (use a join and IS NULL, not a subquery).
5. Subqueries, CTE and set operations.
    a. Return `title` and `price` of books that cost more than the average price of all books (do not type the average), highest price first.
    b. Using EXISTS, return the `author_name` of authors that have at least one book priced above 33, in alphabetical order.
6. Aggregation and window functions.

(a) For books with more than 300 pages, return `author_id`, `total_books` and `avg_price` (rounded to 1 decimal) for each author, but only for authors with at least 2 such books; highest average first.

(b) Return `title`, `price`, `row_num`, `rnk` and `dense_rnk` for all books, ranked library-wide by price from highest to lowest (`row_num` breaks ties by title); order by price DESC, title.